Problem Set on Tutorial on Numpy, Scipy and Astropy

I hope you guys have seen the lecture and also the companion notebook. In this notebook there are few problems and some hints on how to approach them. This problems are not solely on Astronomy. They contains varity of things but all of these are needed to go further in Astronomy and Cosmology.

So, let's begin

In [1]:
```
from jupyterquiz import display_quiz
from IPython.display import HTML
```

In [2]:
```
# @hidden
git_path="https://raw.githubusercontent.com/aburousan/Intro2Astro/main/"
# ============================

# Necessary script to hide the cell:
# ============================
HTML('''<script>
  code_show=true;
  function code_toggle() {
    if (code_show){
        $('.cm-comment:contains(@hidden)').closest('div.input').hide();
    } else {
        $('.cm-comment:contains(@hidden)').closest('div.input').show();
    }
    code_show = !code_show
  }
  $( document ).ready(code_toggle);
</script>''')
```

Out[2]:

In [3]:
```
import numpy as np
import matplotlib.pyplot as plt
import scipy
```

Question-1

Write a Simple python function to calculate **Escape Velocity** of any celestrial body. Calculate the value for earth.

Hints

▶ Forget what is Escape velocity?

In [4]:
```
def v(M,R):
    G = 6.67e-11
    return np.sqrt((2*G*M)/R)
```

In [5]:
```
m = 5.97219e24 # in Kg
r = 6.378e6 # in m
v(m,r) # in m/sec
```

Out[5]: np.float64(11176.4136051077)

In [6]:
```
display_quiz(git_path+"question1.json")#These are for interactive questions
```

Enter the value of v upto 2 decimal places.

Type numeric answer here: 11176.41

Correct.

Can you show why Oxygen molecules cannot go out of earth's atmosphere? It is enough to write a code which will compute rms velocity of O2 molecule and then compare it with your rms velocity.

In [7]:
```
def v_rms(T,M):
    R = 8.314 # j/mol.k
    return np.sqrt((3*R*T)/M)

v_rms(300,0.032) # in m/sec
```

Out[7]: np.float64(483.5610095944461)

The V_rms(Root mean square velocity) of oxygen molecule is far less then the escape velocity of the Earth. This is the reason the oxygen molecule don't go out of earth's atmosphere.

Question-2

Write a function which can calculate the value of π using random numbers using **numpy** library. Try using 1000 sample

In [8]:
```
import numpy as np

def estimate_pi(n_samples=1000):
    x = np.random.rand(n_samples)
    y = np.random.rand(n_samples)
    inside_circle = (x**2 + y**2) <= 1
    pi_estimate = 4 * np.sum(inside_circle) / n_samples
    return round(pi_estimate, 4)  # Round to 4 decimal places

# Run the function with 1000 samples
print(f"Estimated value of π: {estimate_pi(1000):.4f}")
```

```
Estimated value of π: 3.1360
```

In [9]:
```
display_quiz(git_path+"question2.json", colors='fdsp') #These are for interactiv
```

Enter the value of π upto 4 decimal places.

Type numeric answer here: 3.1360

Hey!... I know you can't get all correct digits. You are lucky, I can't use meme here.

Question-3

Sum together every number from 0 to 10000 except for those than can be divided by 4 or 7. Do this using numpy.

In [10]:
```
arr = np.arange(0, 10001)

# Create mask for numbers NOT divisible by 4 or 7
mask = (arr % 4 != 0) & (arr % 7 != 0)

# Apply mask and sum the result
result = np.sum(arr[mask])

print(result)
```

```
32147142
```

In [11]:
```
display_quiz(git_path+"question3.json", colors='fdsp')#These are for interactiv
```

What is the sum?

Type numeric answer here: 32147142

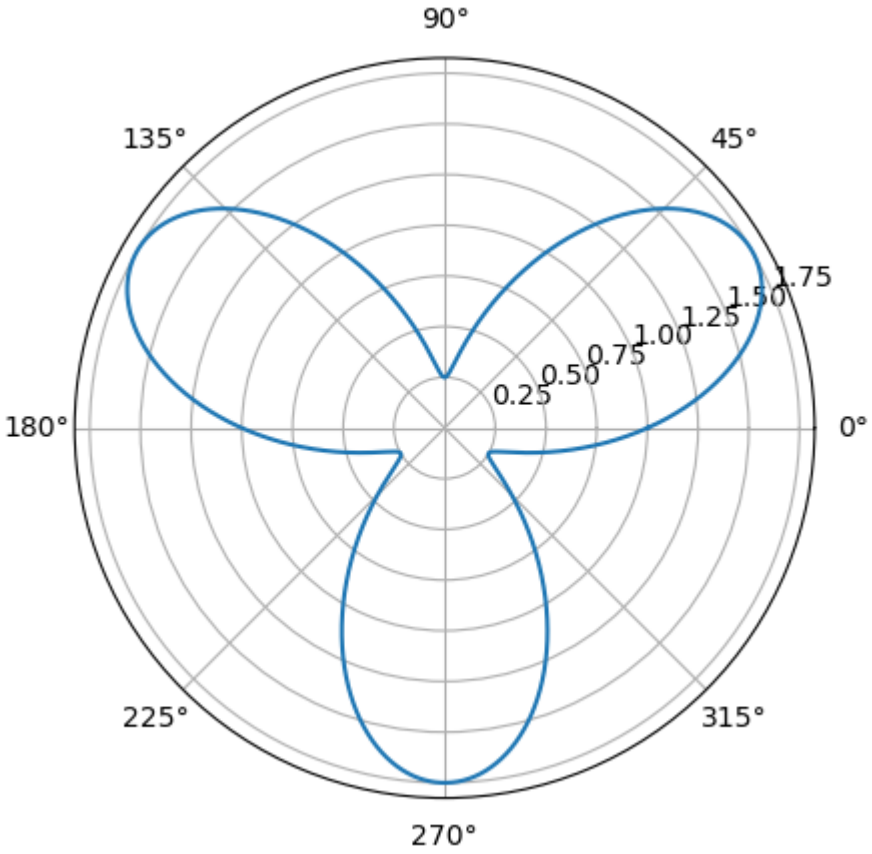
Correct.

Question-4

Consider the flower petal $r(\theta) = 1 + \frac{3}{4}\sin(3\theta)$ for $0 \leq \theta < 2\pi$.

1. Plot the shape.
2. Compute the area. If you guys don't know the formula. It is $A = \int_0^{2\pi} \frac{r^2}{2} d\theta$

In [12]:
```
theta = np.linspace(0,2*np.pi,100000)
r = 1+((3/4)*np.sin(3*theta))
plt.polar(theta, r)
plt.show()
```



In [13]:
```python
from scipy.integrate import quad

# Define the function r(θ)
def r(theta):
    return 1 + (3/4) * np.sin(3 * theta)

# Define the integrand for area
def integrand(theta):
    return 0.5 * r(theta)**2

# Integrate from 0 to 2π
area, error = quad(integrand, 0, 2 * np.pi)

print(f"Area enclosed by the polar curve = {area:.4f}")
```

```
Area enclosed by the polar curve = 4.0252
```

In [14]:
```python
display_quiz(git_path+"question4.json", colors='fdsp')#These are for interact
```

What is the area you are getting? Write upto 4 decimal places

Type numeric answer here: 4.0252

TRy aggain... You are lucky, I can't use meme here.

Not a question but a suggestion.

Try solving any KVL or KCL problem using numpy.(Linear equation solution)

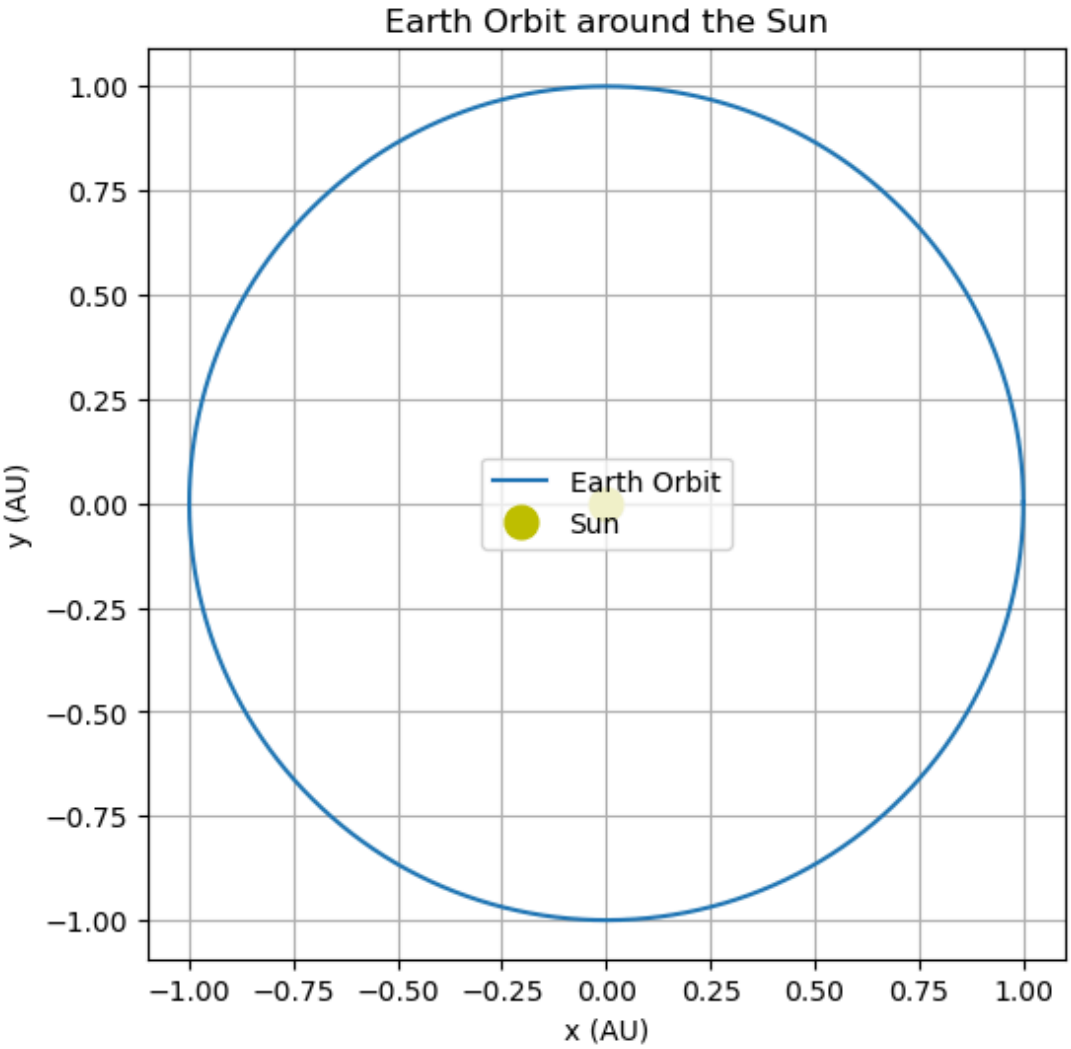
Question-5

Use Newton's Gravitational Law along with Newton's 2nd law of motion to write the differential equation which earth will follow due to Sun's gravity (sun is fixed in it's place).

1. Now use Scipy to solve the equation. The constants needed must be imported from Astropy.
2. Plot your solution , i.e., x-y plot.

In [15]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
from astropy import constants as const
from astropy import units as u

# Constants
G = const.G.value                # gravitational constant in m^3 kg^-1 s^-2
M_sun = const.M_sun.value        # mass of Sun in kg

# Define the equations of motion
def two_body(t, y):
    x, vx, y_pos, vy = y  # unpack
    r = np.sqrt(x**2 + y_pos**2)
    ax = -G * M_sun * x / r**3
    ay = -G * M_sun * y_pos / r**3
    return [vx, ax, vy, ay]

# Initial conditions
AU = const.au.value  # 1 Astronomical Unit in meters
x0 = AU
y0 = 0.0
vx0 = 0.0
vy0 = 29_780  # m/s, Earth's orbital speed

# Initial state vector: [x, vx, y, vy]
y_init = [x0, vx0, y0, vy0]

# Time span for one year
t_span = (0, 365.25 * 24 * 3600)  # seconds in a year
t_eval = np.linspace(t_span[0], t_span[1], 2000)

# Solve the ODE
sol = solve_ivp(two_body, t_span, y_init, t_eval=t_eval, rtol=1e-8)

# Extract results
x = sol.y[0]
y = sol.y[2]

# Plot orbit
plt.figure(figsize=(6, 6))
plt.plot(x / AU, y / AU, label='Earth Orbit')
plt.plot(0, 0, 'yo', markersize=12, label='Sun')  # Sun at origin
plt.xlabel('x (AU)')
plt.ylabel('y (AU)')
plt.title('Earth Orbit around the Sun')
```

```
plt.grid(True)
plt.axis('equal')
plt.legend()
plt.show()
```



Question-6

NASA Cosmic Background Explorer (COBE) satellite carried an instrument, **FIRAS** (Far-Infrared Absolute Spectrophotometer) to measure the cosmic microwave background (CMB) radiation, which was confirmed to be distributed according to a black-body curve in accordance with the big bang theory:

$$I(\nu, T) = \frac{2h\nu^3c^2}{\exp(\frac{hc\nu}{k_bT}) - 1}$$

where where the radiation frequency is expressed in wavenumbers, cm^{-1}, and the speed of light, c is taken to be in $cm - s^{-1}$.

The data file is cmb_data.txt, which contains measured $I(\nu)$ based on the FIRAS observations. Use scipy `curve_fit` to determine T, i.e., the Temperature parameter, along with error.

Note: In the file I is in $erg \cdot s^{-1} \cdot cm^{-1} \cdot sr^{-1}$. Take the estimated σ error in the measurement to be $2 \times 10^{-6} erg \cdot s^{-1} \cdot cm^{-1} \cdot sr^{-1}$.

In [16]:
```
from astropy import constants as const
from astropy import units as u
from scipy.optimize import curve_fit

h = const.h.to('erg.s').value
k = const.k_B.to('erg/K').value
c = const.c.to('cm/s').value

data = np.loadtxt('cmb_data.txt')
u = data[:,0]
i = data[:,1]

def I(u,T):
    n = 2*h*c**2 * u**3
    d = (h*c*u)/(k*T)
    return n/(np.exp(d)-1)

popt, pcov = curve_fit(I,u,i, p0=[2.7])

T_fit = popt[0]
T_err = np.sqrt(np.diag(pcov))[0]

print(f"Fitted Temperature: {T_fit:.4f} K ± {T_err:.4f} K")

#para, pcov = curve_fit(x, y, fit_func, p0=(T0,), sigma=σ, absolute_sigma=Tr
```

```
Fitted Temperature: 2.7147 K ± 0.0041 K
```

In [17]:
```
display_quiz(git_path+"question6.json", colors='fdsp')#These are for interac
```

What is the value of T you are getting?

Type numeric answer here: 2.7147

Happy Happy dancing cat...

What is the error?

Type numeric answer here: 0.004

Ghost in your room congratulating you!

Question-7

Calculate the rest mass energy of a Proton in both joule and MeV.

If you want to get the list of constants present in AstroPy. Check the bottom of this link:https://docs.astropy.org/en/stable/constants/index.html

In [18]:
```
m = const.m_p
c = const.c

E_rest = m*c**2

E_J = E_rest.to('Joule')
E_Mev = E_rest.to('MeV')
print(f"The rest mass of proton is {E_J}")
print(f"The rest mass of proton is {E_Mev}")
```

```
The rest mass of proton is 1.5032776159851256e-10 J
The rest mass of proton is 938.2720881604905 MeV
```

In [19]:
```
display_quiz(git_path+"question7.json", colors='fdsp')#These are for interc
```

What's the value of E_0 of a proton ... you are getting?(in MeV)

Type numeric answer here: 938.27

Failure!

What's the value of E_0 of a proton ... you are getting?(in MeV)

1.4533 X 10^{-10} J

1.5033 X 10^{-10} J

1.6033 X 10^{-10} J

Correct but in batman voice

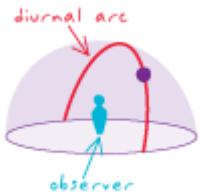
Question-8

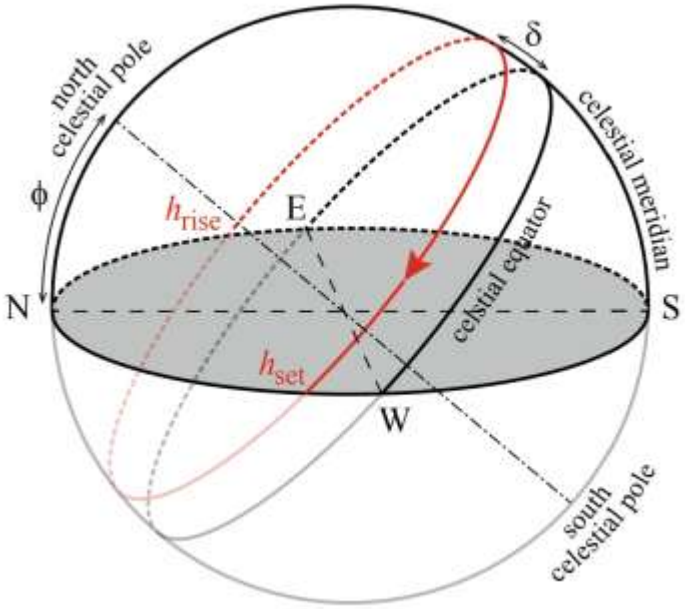
Theory needed for problem-8

Diurnal motion is an astronomical term referring to the apparent motion of celestial objects (e.g. the Sun and stars) around Earth, or more precisely around the two celestial poles, over the course of one day.

It is caused by Earth's rotation around its axis, so almost every star appears to follow a circular arc path, called the diurnal circle,often depicted in star trail photography.

So, From the viewpoint of an observer on Earth, the apparent motion of an object on the celestrial sphere follows an arc above the horizon, which is called **diurnal arc**.





Here, Diurnal Arc of a Star moving around the celestrial sphere(red line) in the horizontal system of an observer at lattitude ϕ. Since, the equatorial plane is inclined by the angle $90^\circ - \phi$ against the horizontal plane, the upper culmination of the star at the meridian is given by $a_{max} = 90^\circ - \phi + \delta$, where δ is the declination. The star **rises** at hour angle h_{rise}, reaches it's highest altitude when it crosses the meridian at $h = 0$ and sets at the horizon at $h_{set} = -h_{rise}$. The value can be given by,

$$\cos(h_{rise}) = -\tan(\delta)\tan(\phi)$$

Sidereal Time is the time for which the star is visible on sky. It is given by $T = 2h_{set}$.

Problem-8: Find how long the star `Betelgeuse` is present on sky from my location (Jadavpur, Kolkata, India).

In [20]:
```
from astropy.coordinates import SkyCoord, EarthLocation
import astropy.units as u
HG = SkyCoord.from_name('Betelgeuse')
print(HG)
del_hg = HG.dec
print(del_hg)
```

```
obs = EarthLocation(lon=88*u.deg + 22*u.arcmin+23.88*u.arcsec,
                    lat=22*u.deg + 29*u.arcmin+55.32*u.arcsec)
phi = obs.lat
print(phi)
```

```
<SkyCoord (ICRS): (ra, dec) in deg
    (88.79293899, 7.40706399)>
7d24m25.430382s
22d29m55.32s
```

In [21]:
```
#Now, calculate h
import math as m
h = m.acos(-m.tan(del_hg.radian)*m.tan(phi.radian))
print("h = ",h)
T = (m.degrees(2*h)/360)*u.day #conversion between sidereal and solar day
T_in_h = T.to(u.h)
print("T = ",T_in_h)
```

```
h =  1.6246678039210198
T =  12.411547770061656 h
```

In [22]:
```
display_quiz(git_path+"question8.json", colors='fdsp')#These are for inte
```

What is the value you are finding for T of Betelgeuse?

Type numeric answer here: 12.41145

Why man why?, You just had to change a variable name.

Question-9

The **Declination** of sun δ_s is given by,

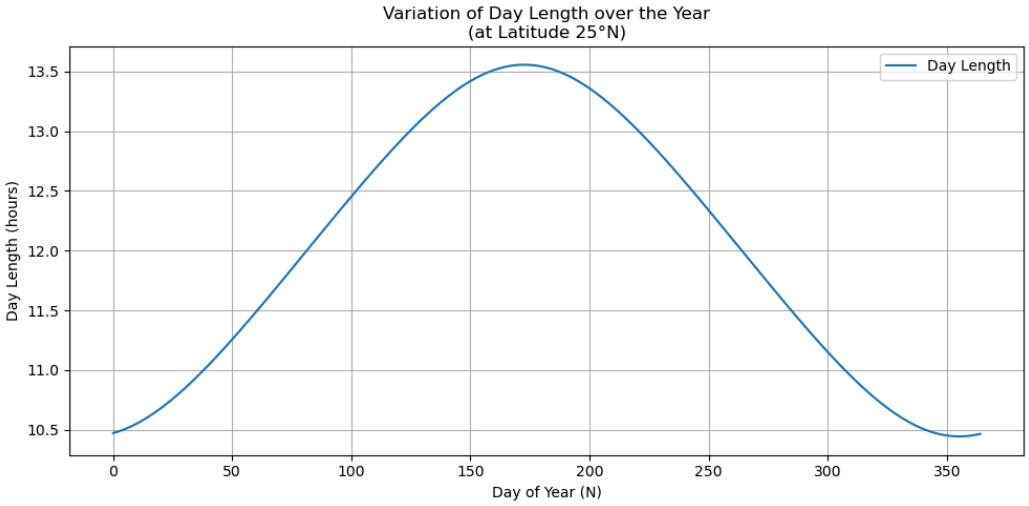
$$\delta_s = -\arcsin\left(\sin(\epsilon_0)\cos\left(\frac{360}{365.24}(N+10)\right)\right)$$

where $\epsilon_0 = 23.44^\circ$ and N is the difference in days starting from 1st january.

Make a plot of how the length of day changes over the year in your location.

In [23]:
```
import numpy as np
import matplotlib.pyplot as plt

# Constants
epsilon = np.radians(23.44)  # axial tilt in radians
days = np.arange(0, 365)     # N: days from Jan 1 to Dec 31
phi = np.radians(25.0)       # replace 25 with your latitude in degrees

# Declination formula
decl = -np.arcsin(np.sin(epsilon) * np.cos(2 * np.pi * (days + 10) / 365

# Calculate day length in hours using the hour angle
```

```
# Avoid invalid values due to extreme latitudes
cos_omega = -np.tan(phi) * np.tan(decl)
cos_omega = np.clip(cos_omega, -1, 1)

# Hour angle (in degrees) converted to time (in hours)
day_length = 2 * np.degrees(np.arccos(cos_omega)) / 15  # 15° = 1 hour

# Plotting
plt.figure(figsize=(10, 5))
plt.plot(days, day_length, label='Day Length')
plt.xlabel('Day of Year (N)')
plt.ylabel('Day Length (hours)')
plt.title('Variation of Day Length over the Year\n(at Latitude 25°N)')
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```



Question-10

Now, let's play with some spectra. The spectra, we are going to use, was obtained at the 2.5m INT telescope and cover the range 3525-7500 Å (Sánchez-Blázquez et al. 2006) at 2.5 Å (FWHM) spectral resolution (Falcón-Barroso et al. 2011).

Plot the spectra from the fit file.

Hints

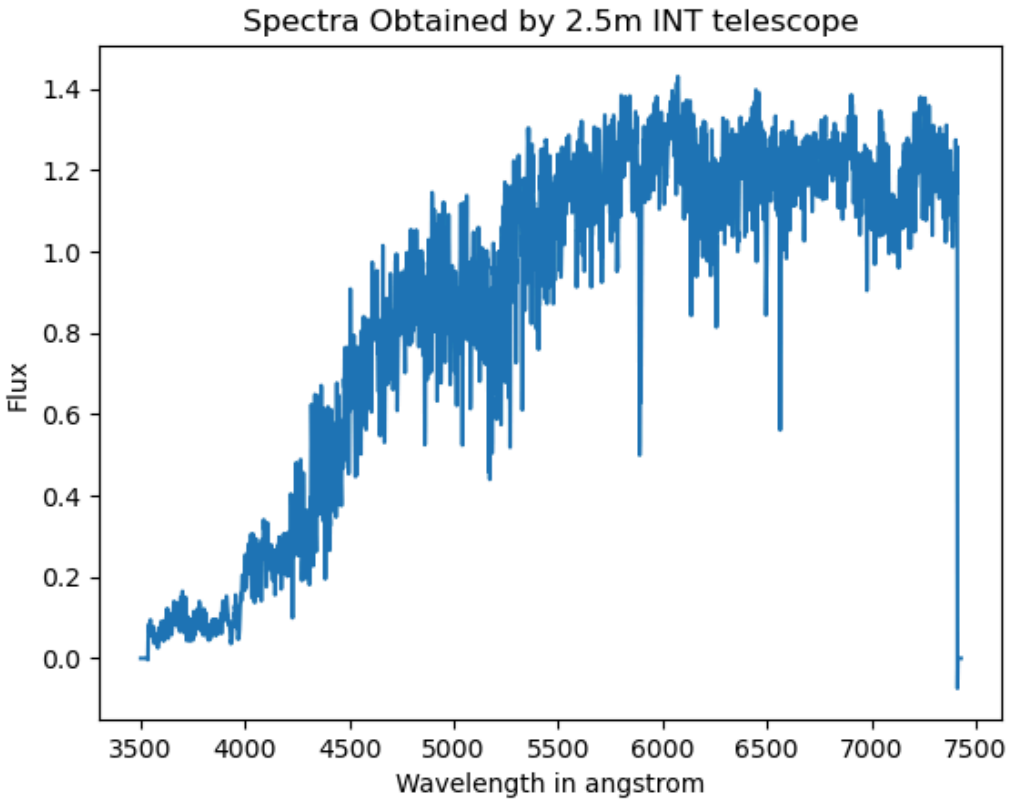
▶ How to get the wavelength?

In [24]:
```
from astropy.io import fits
import matplotlib.pyplot as plt
from astropy.wcs import WCS

# Load the FITS file
hdul = fits.open("s0002.fits")
data = hdul[0].data
h1 = hdul[0].header
print(repr(h1))
```

```
obj_name = h1.get('OBJECT', 'Unknown')
flux = data[0]
w = WCS(h1,naxis=1,relax=False,fix=False)
wv_am = w.wcs_pix2world(np.arange(len(flux)), 0)[0]

plt.plot(wv_am,flux)
plt.xlabel('Wavelength in angstrom')
plt.ylabel('Flux')
plt.title('Spectra Obtained by 2.5m INT telescope')
```

```
SIMPLE  =                    T / file does conform to FITS standard
BITPIX  =                  -32 / number of bits per data pixel
NAXIS   =                    2 / number of data axes
NAXIS1  =                 4367 / length of data axis 1
NAXIS2  =                    1 / length of data axis 2
COMMENT   FITS (Flexible Image Transport System) format is defined in 'As
tronomy
COMMENT   and Astrophysics', volume 376, page 359; bibcode: 2001A&A...37
6..359H
COMMENT ---------------------------------------------------------------
-----
COMMENT ************************ REDUCEME HEADER ********************
*****
COMMENT ---------------------------------------------------------------
-----
HISTORY Date: 06/10/**
CRPIX1  =                 1.00
CRVAL1  =            3500.0000 / central wavelength of first pixel
CDELT1  =             0.900000 / linear dispersion (Angstrom/pixel)
OBJECT  = 'HD225212'           / Object name
FITSFI  = 's0002.fits'         / FITS file name
AIRMASS =              0.00000 / Airmass
TIMEXPO =               -999.0 / Timexpos
```

Out[24]: Text(0.5, 1.0, 'Spectra Obtained by 2.5m INT telescope')



https://classic.sdss.org/dr6/algorithms/linestable.html This link contains wavelengths and their corresponding element. Check from here.

In [25]:
```
display_quiz(git_path+"question10.json", colors='fdsp')#These are for ir
```

Is there Na absorption line in the plot?

Yes

No

No Idea

Correct.. I am not going to make a Sodium joke... don't worry